Master 1 in Computer Science (Informatique), ISI
Image Processing mini-project

A Comparative Study of RAG Techniques for Image Captioning on MS COCO

Phase 1 report: Data Exploration and Preprocessing

Prepared by:

Taleb Amine (G4)
Taleb Omar (G4)
Kasraoui Mohamed Ali (G3)
Missouni Aimen (G4)

Repository: github.com/san9r/image-processing-

November 2026

Abstract

This report presents the first phase of our project, which consists in preparing the data used in all the later experiments. We built a knowledge base (KB) of 10,000 COCO images, a validation set of 100 images and a test set of 1,000 images. A smaller test set of 100 images, taken from the large one, is kept for comparison. For these images we prepared the captions, computed CLIP embeddings, built a FAISS index and extracted the objects, first from the COCO annotations and then directly from the images with the YOLOv8n detector.

The confidence threshold of the detector was chosen on the validation set (0.25), and the statistical study is split in two parts, before and after preprocessing. Our audit script runs 34 checks and none of them fails. No captioning model is evaluated in this phase.

Keywords: image captioning, RAG, MS COCO, CLIP, FAISS, YOLO, preprocessing.

1. Introduction

Image captioning models are able to write fluent descriptions, but they still make mistakes. They can miss objects, or mention objects that are not in the image, which is called hallucination. Retrieval-Augmented Generation (RAG) is a way to reduce these mistakes: before writing a caption, the system looks for similar images and their captions in a knowledge base and uses them as a help.

The project compares several RAG techniques on the same data. For the comparison to be fair, all the groups need the same images, the same splits and the same index. The goal of Phase 1 is to build this common base and to check it. No caption is generated at this stage.

2. Objectives of Phase 1

In this phase we had to:

- choose the images and create three separate sets (knowledge base, validation and test);
- make sure that no evaluation image or caption can end up in the knowledge base;
- prepare the captions, keeping the originals and making a cleaned copy for the search;
- extract the objects of each image and document how it was done;
- compute the embeddings and build the index;
- describe the data with statistics and figures, before and after preprocessing;
- check all the outputs automatically.

3. Tools

The dataset is MS COCO 2017. It provides the images, five captions per image written by humans, and object annotations for 80 categories. The KB and validation images come from *train2017* and the test images from *val2017*.

To represent the images and the captions we use CLIP ViT-B/32, which turns an image or a sentence into a vector of 512 numbers. Images and captions with a close meaning get close vectors. The vectors are stored in FAISS, with an *IndexFlatIP* index, which does an exact search with the cosine similarity.

The objects are detected with YOLOv8n from the Ultralytics library (version 8.4.208). For each object it gives a category, a confidence score and a box. We also used NLTK for the word types, Matplotlib for the figures, PyTorch to run the models and Pillow to open the images.

4. Dataset and splits

4.1 The three sets

All the scripts use the random seed 42, so the same images are selected each time. The images with fewer than five captions are not used. Table 1 gives the sets.

Table 1: The data splits

Split	Source	Images	Captions	Used for
Knowledge base	COCO train2017	10,000	50,000	What the RAG system searches in
Validation	COCO train2017, not in the KB	100	500	Choosing the settings
Test	COCO val2017	1,000	5,000	Final evaluation
Test (small)	Part of the 1,000 test images	100	500	Comparison with the full test set

4.2 Building the test set

The test set has to contain simple scenes and complex scenes. We took the *val2017* images which have at least one annotated object, sorted them by number of objects and cut the list into three equal groups: simple scenes (1 to 3 objects), medium scenes (3 to 8) and complex scenes (8 to 63). About one third of the test images is taken from each group, which gives 334, 333 and 333 images.

The script *validation.py* first selects 100 test images in this way. The script *split_test1000.py* keeps them and adds new images in each group until there are 1,000. So the small test set is a part of the large one, and both can be used to compare results.

4.3 Avoiding leakage

If a test image was also in the knowledge base, the system could just find its own captions and the results would mean nothing. To avoid this:

- the validation images are taken only among the *train2017* images which are not in the KB;
- the test images come from *val2017*, which is a different part of COCO;
- the index contains only KB images and KB captions;
- the scripts and the audit check that no image is in two sets.

5. Preprocessing pipeline

The work is done by 15 scripts. They are run in the order of Table 2.

Table 2: The scripts in the order they are run

#	What it does	Script	Main output
1	First look at the COCO files	explore.py	Printed counts
2	Selects the 10,000 KB images	split.py	splits/kb_train_ids.json
3	Selects the validation set and the first test set	validation.py	splits/val_ids.json, splits/test_ids.json
4	Builds the 1,000-image test set	split_test1000.py	splits/test_ids_1000.json
5	Downloads the images	images.py	images/kb, images/val, images/test
6	First statistics (before preprocessing)	stats.py	Three figures
7	Prepares the captions	preprocess.py	processed/*.json
8	Computes the embeddings	embeddings.py	.npy files, embedding_info.json
9	Builds the indexes	build-index.py	.index files, index_info.json
10	Reads the human object labels	extract_metadata.py	metadata/kb_metadata.json, *_objects_gt.json
11	Detects the objects in the images	detect_objects.py	metadata/detected_*.json
12	Statistics on captions and human labels	stats2.py	reports/dataset_stats.json, six figures
13	Threshold choice, statistics on detected objects	stats3.py	reports/dataset_stats_detected.json, five figures
14	Comparison of the sets, scatter plot, positions	stats4.py	reports/dataset_stats_extra.json, four figures
15	Checks everything	audit.py	reports/qc_report.json

5.1 Images

The images are downloaded from the links given in the COCO files. Before the embedding is computed, each image is put in its correct orientation and converted to RGB. The resizing is done by CLIP and YOLO themselves.

5.2 Captions

We keep exactly five captions per image, in two versions. The first one, *caption_raw*, is the original caption and is used for the evaluation. The second one, *caption_clean*, is in lower case without punctuation and is used only for the search. The validation and test images do not have a cleaned version, so the evaluation cannot use a cleaned caption by mistake.

5.3 Objects

The objects of each image come from two sources, saved in separate files.

The first source is the detector. YOLOv8n was run on the 11,100 images, and each detected object is saved with its category, its confidence score (between 0 and 1) and its box. We saved all the detections with a confidence of at least 0.25. The detector found 51,395 objects in the KB, 608 in the validation set and 5,049 in the test set.

The second source is the COCO annotations, which were written by people. They count as confidence 1 and do not need a threshold. They are saved for the KB, the validation set and the small test set.

5.4 Choice of the confidence threshold

The detector gives a confidence score to each object, and we have to decide from which score an object is kept. We made this choice on the 100 validation images. For each threshold, we compared the categories found by the detector with the categories of the human labels. The precision is the share of the reported categories which are really in the image, the recall is the share of the real categories which were found, and F1 combines the two. The results are in Table 3 and Figure 1.

Table 3: Detector compared with the human labels on the 100 validation images

Threshold	Precision	Recall	F1
0.25 (chosen)	0.863	0.724	0.787
0.30	0.884	0.708	0.786
0.35	0.900	0.671	0.769
0.40	0.911	0.637	0.750
0.45	0.933	0.602	0.732
0.50	0.962	0.556	0.705
0.55	0.977	0.519	0.677
0.60	0.988	0.491	0.656
0.70	0.992	0.404	0.574

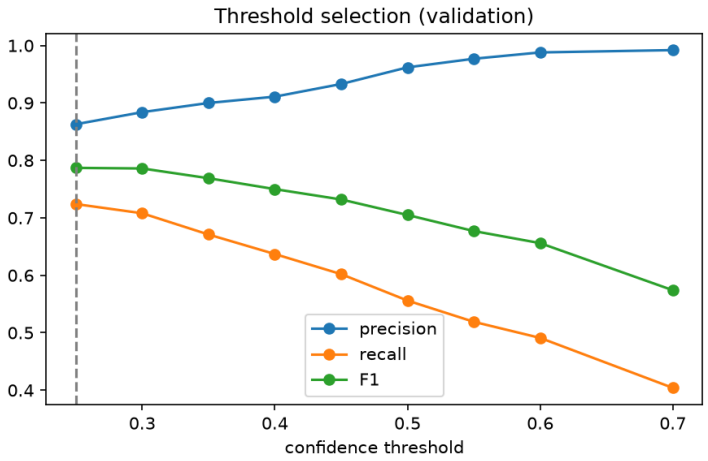


Figure 1: Precision, recall and F1 for each threshold (the dashed line is the chosen value)

When the threshold goes up, there are fewer wrong objects but more missed objects. The best F1 (0.787) is obtained at 0.25, so we kept this value. With 0.30 the result is almost the same (0.786). We could not try values under 0.25 because these detections were not saved.

6. Statistical exploration

This section describes the 10,000 images of the knowledge base. Section 6.1 concerns the data before preprocessing (original captions and human labels), and Sections 6.2 to 6.5 the data after preprocessing (cleaned captions and detected objects).

6.1 Before preprocessing

The human labels contain 72,062 objects in 80 categories. A few categories are very frequent and most of them are rare. The category *person* is by far the first one (22,085 objects), followed by *car* (3,619), *chair* (3,180), *bottle* (2,045) and *book* (2,003). An image has 7.21 objects on average (median 4, maximum 56) and 102 images have no labelled object. These distributions are shown in Figures 4 and 5, next to the detected objects.

We also looked at which categories appear in the same image (Figure 7). Most of the frequent pairs include a person: person and car (692 images), person and chair (671), person and handbag (510), person and dining table (495).

For the text, the KB has 50,000 captions. A caption has 10.47 words on average (median 10, between 6 and 46 words). Table 4 and Figure 2 give the types of words. Captions are mostly made of nouns and adjectives are rare, which means that captions say what is in the image but give few details like colours.

Table 4: Types of words in the KB captions

Type of word	Share (%)	Type of word	Share (%)
Noun	36.05	Function word	5.97
Determiner	20.48	Number	1.24
Preposition	15.96	Adverb	0.65
Verb	11.54	Other	0.20
Adjective	7.90		

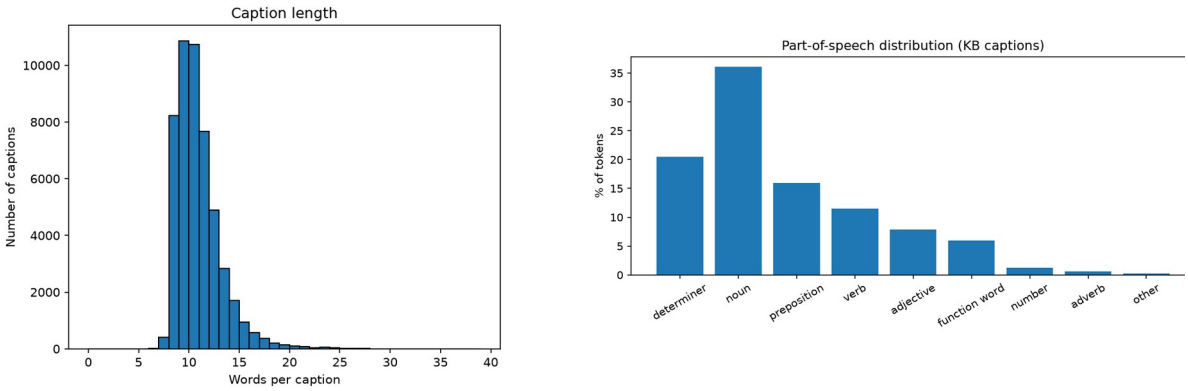


Figure 2: Number of words per caption (left) and types of words (right)

COCO has no labels for colours, shapes, positions or context, so we counted words in the captions (Figure 3). These numbers describe the captions and not the images themselves. The most frequent colours are *white* (2,966 captions), *black* (1,675), *red* (1,397), *blue* (1,128) and *green* (1,044). Shape words are rare: *tall* (360) and *long* (276) appear more than *square* (70) or *round* (53). For the positions, *on* (17,047) and *in* (14,748) are very frequent, then *at* (3,348),

next to (3,202) and *near* (1,670). Finally, 4,323 images have an outdoor word, 2,065 an indoor word, 448 a day word and 258 a night word; most captions do not say if it is day or night.

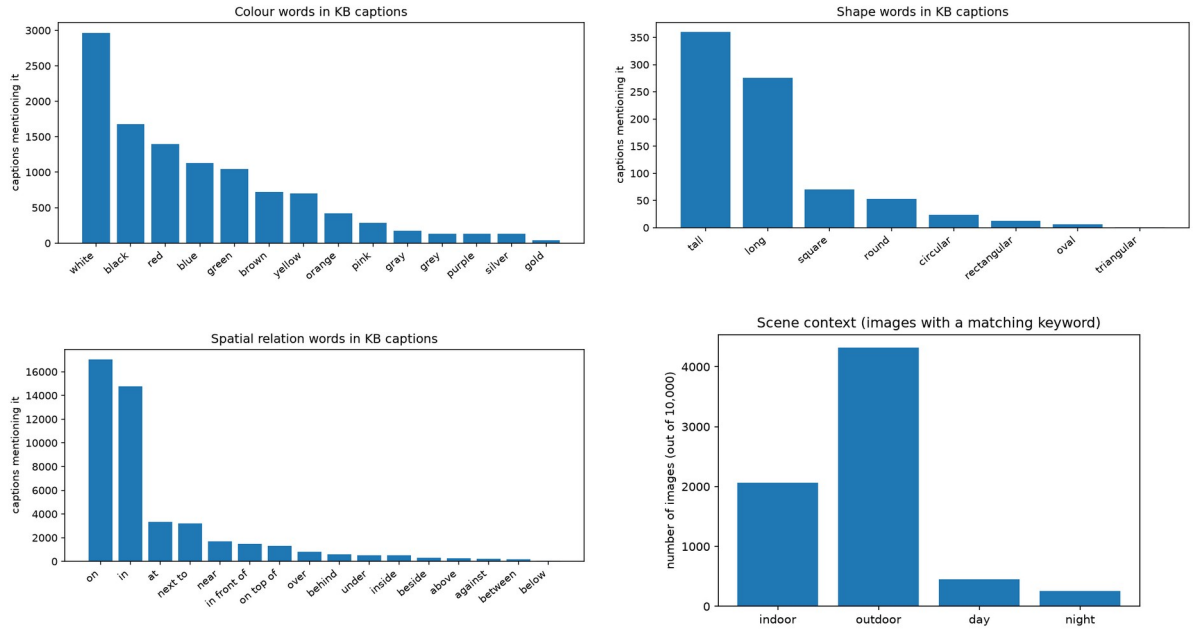


Figure 3: Words counted in the KB captions: colours, shapes, positions and context

6.2 After preprocessing

With the threshold of 0.25, the detector finds 51,395 objects in the KB, which is 5.14 per image on average (median 4, maximum 44). In 160 images it finds nothing. It finds 79 of the 80 categories (*hair drier* is never detected). The most frequent ones are *person* (17,980), *car* (2,443), *chair* (1,785), *bottle* (1,338) and *cup* (1,092). The order is almost the same as with the human labels (Figure 4).

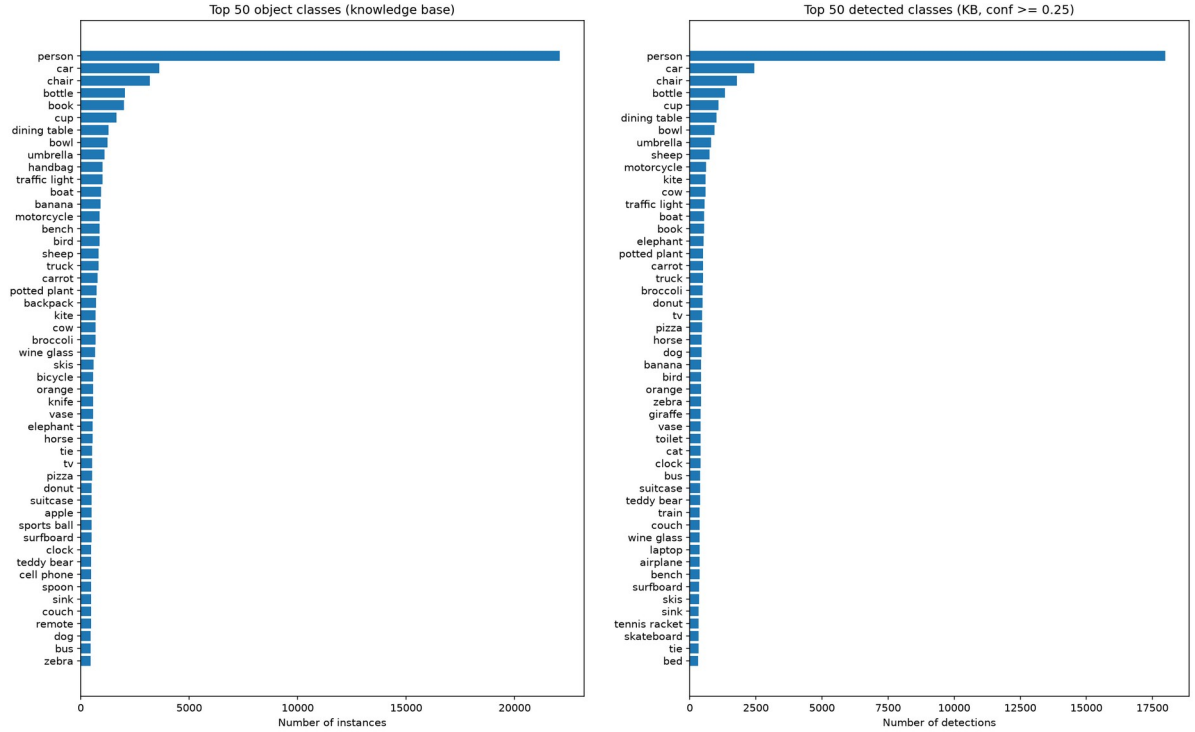


Figure 4: The 50 most frequent categories in the KB: human labels (left) and detected objects (right)

The detector finds fewer objects than the human annotators: 51,395 against 72,062, so about 71%. The difference is bigger in crowded images (Figure 5) and for small objects. For example it finds only about a quarter of the *book* and *handbag* objects (Figure 6). This was expected, since we use the smallest YOLO model.

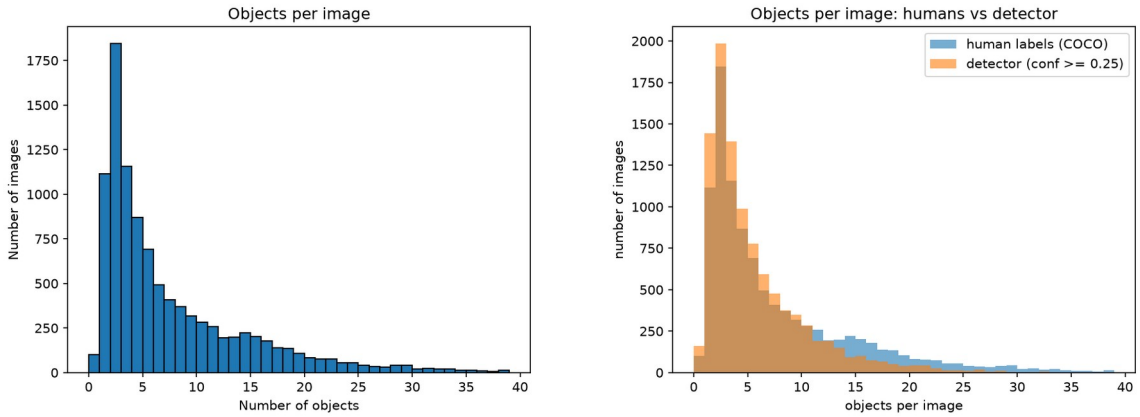


Figure 5: Objects per image in the KB: human labels (left) and human labels compared with the detector (right)

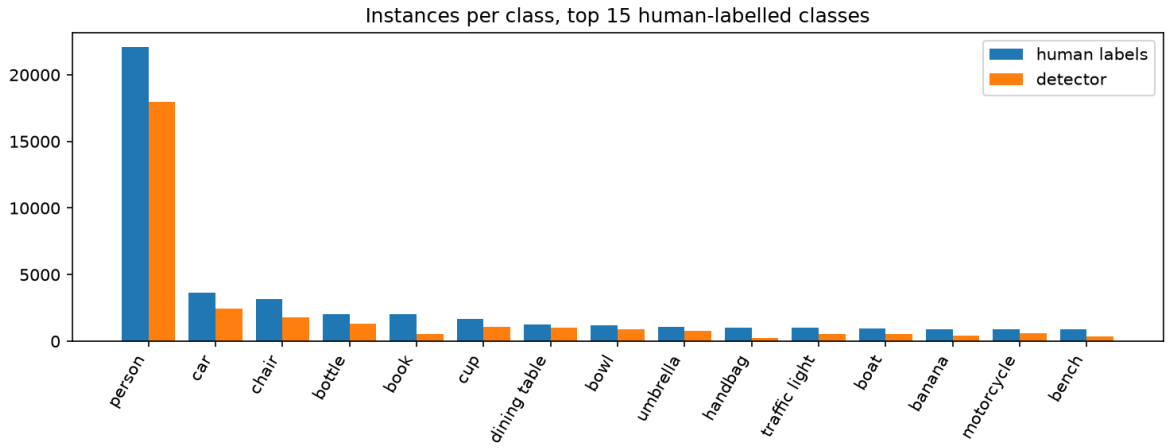


Figure 6: Number of objects in the 15 most frequent categories, human labels and detector

The categories which appear together are the same with the detected objects as with the human labels (Figure 7). *person* appears with almost all the categories, and *dining table*, *chair*, *cup*, *bottle* and *bowl* are often together.

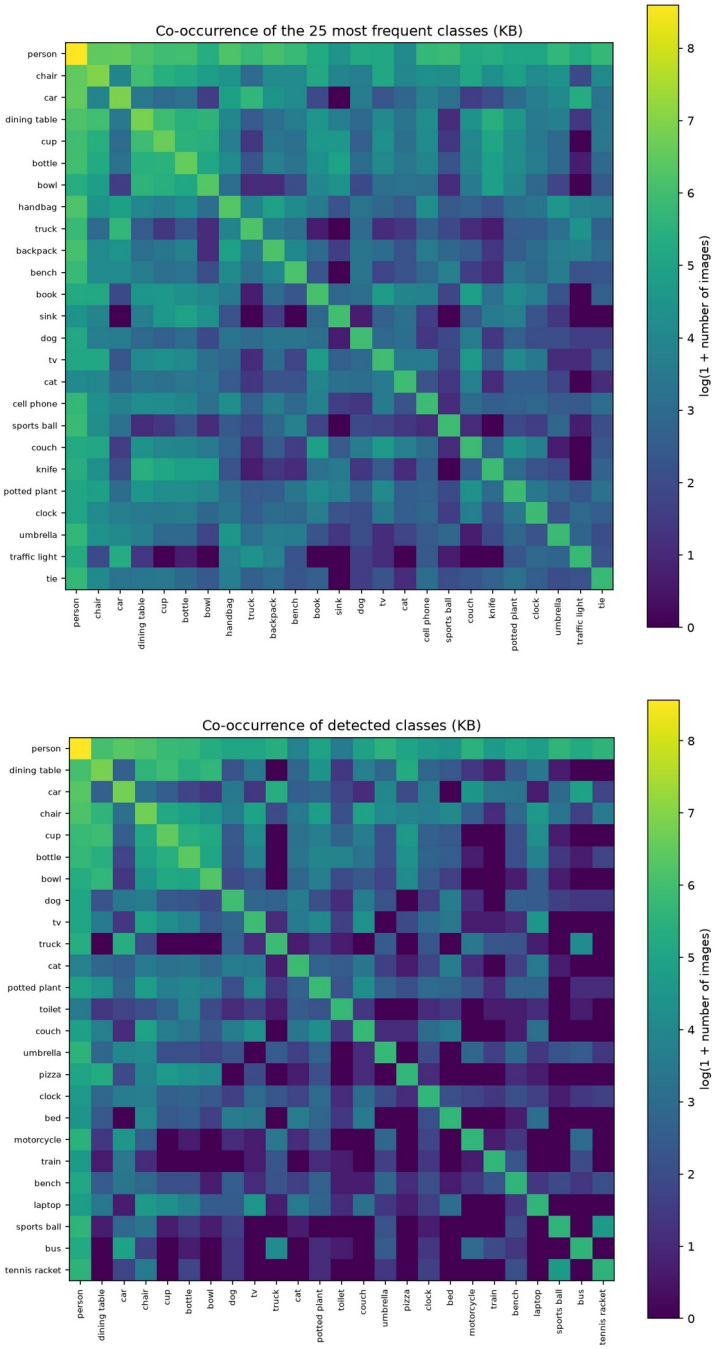


Figure 7: How often two categories are in the same image, for the 25 most frequent categories: human labels (top) and detected objects (bottom)

Table 5 compares the captions before and after cleaning. The cleaning does not remove any word, so the length is the same. The number of different words goes down by 39% because forms like *Dog*, *dog* and *dog.* become a single word, which helps the search.

Table 5: KB captions before and after cleaning

	Average length (words)	**Different words**
Original captions	10.47	16,228
Cleaned captions	10.47	9,917

6.3 Positions of the objects, from the boxes

Counting words like *on* or *in* does not really say where the objects are. For this reason we used the boxes of the detected objects. For each KB image we took all the pairs of objects (with at most the 20 most confident objects) and looked at how the two boxes are placed.

On 200,492 pairs, 60.9% are side by side, 17.2% are one above the other, 15.7% overlap and in 6.1% one box is inside the other (Figure 8). The most frequent "above" pairs are kite above person (1,114), person above chair (878) and umbrella above person (867). These positions are measured in the flat image and not in the real scene: *person above cup*, for example, is usually a person sitting behind a table.

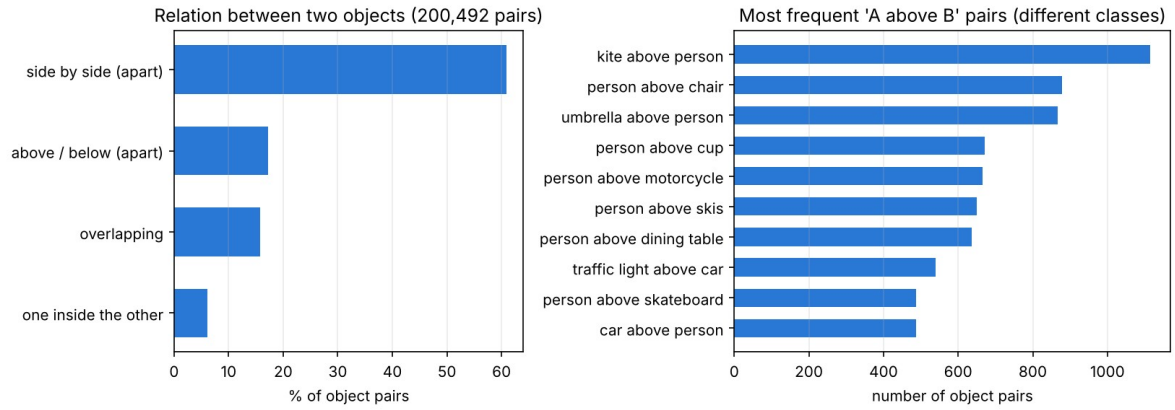


Figure 8: Positions of the pairs of detected objects in the KB, computed from their boxes

6.4 Number of objects and caption length

We wanted to know if images with more objects have longer captions. Figure 9 shows that it is almost not the case. The correlation is very weak (Spearman 0.109, Pearson 0.075), and a caption has between 10 and 11 words whatever the number of objects.

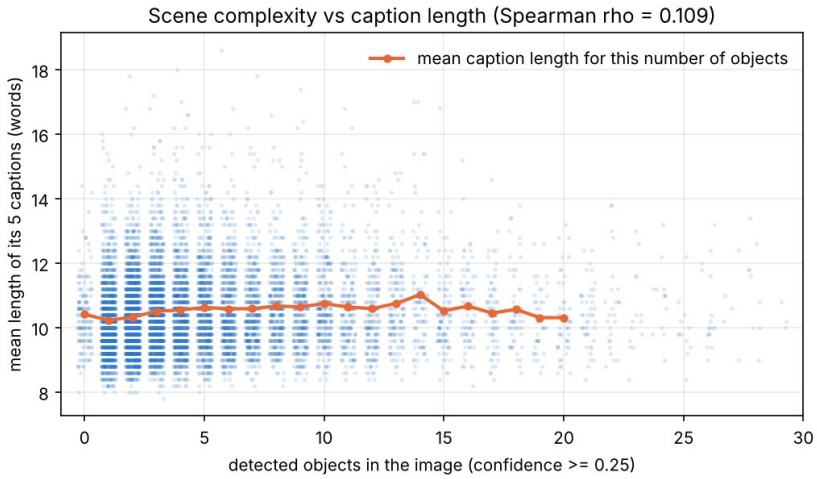


Figure 9: Number of detected objects and average length of the five captions (one point per KB image; the orange line is the average)

A caption summarises the scene and does not list all the objects. The object lists therefore contain information which is not in the captions, and this makes them useful for the retrieval. It also means that we should not expect a generated caption to mention every object.

6.5 Comparison of the three sets

The validation and test sets should look like the knowledge base. Table 6 compares them.

Table 6: The three sets (objects per image are given as median / average / maximum)

Set	Images	Detected objects per image	Human-labelled objects per image	Images with a person (%)	Average caption length
Knowledge base	10,000	4 / 5.14 / 44	4 / 7.21 / 56	52.5	10.47
Validation	100	4 / 6.08 / 24	6 / 8.85 / 43	61.0	10.80
Test	1,000	3 / 5.05 / 33	not extracted	52.9	10.48
Test (small)	100	4 / 4.94 / 21	5 / 7.51 / 30	52.0	10.43

We also compared the sets two by two with the Kolmogorov-Smirnov test (Table 7). A p-value above 0.05 means that the test does not find a real difference. As we do three comparisons, we use the stricter limit 0.05 / 3 = 0.0167.

Table 7: Kolmogorov-Smirnov tests between the sets (p-values)

Comparison	Objects per image	Caption length
Knowledge base and test	0.958 (no difference)	> 0.999 (no difference)
Knowledge base and validation	0.271 (no difference)	0.012 (small difference)
Validation and test	0.192 (no difference)	0.012 (small difference)

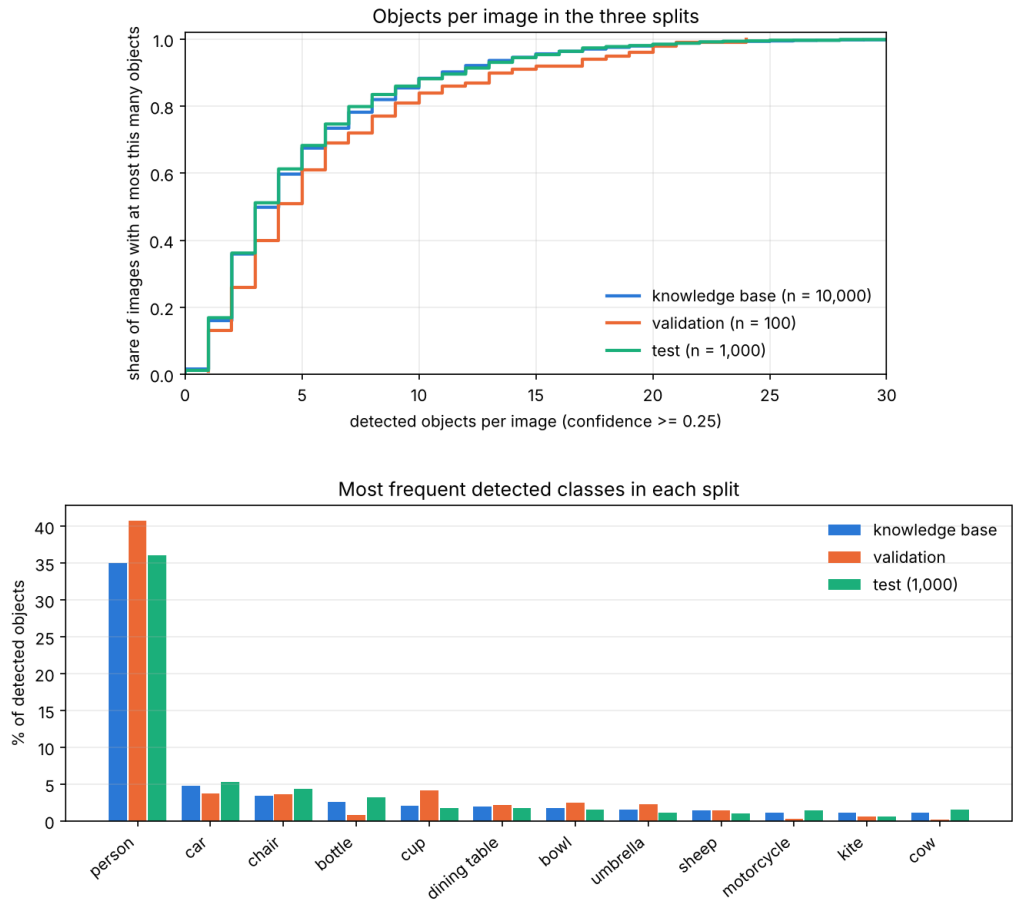


Figure 10: The three sets compared: objects per image (top) and most frequent categories (bottom)

The knowledge base and the test set are very similar, so the test set represents well the images stored in the KB. The validation set is a little different: its captions are slightly longer (10.80 words against 10.47) and its images contain a person more often. This can happen with only 100 images, but we have to remember it when the settings chosen on the validation set are used on the test set.

7. Embeddings and index

Each KB image and each cleaned KB caption is turned into a vector of 512 numbers with CLIP ViT-B/32, and the vectors are normalised to length 1. On our CPU it took 354 s for the 10,000 images and 422 s for the 50,000 captions. The vectors are stored in two FAISS indexes (Table 8). An exact index is fast enough for 10,000 images and makes no search error.

Table 8: The two indexes

	Image index	**Caption index**
Type	IndexFlatIP (exact search)	IndexFlatIP (exact search)
Similarity	Cosine	Cosine
Vector size	512	512
Number of vectors	10,000	50,000
Build time	0.0057 s	0.0301 s
File size	20.48 MB	102.40 MB

To check that the search works, we searched the caption index with 1,000 KB images. The closest caption belonged to the same image in 44.3% of the cases (Recall@1 = 0.443), and one of the five closest captions did in 65.0% of the cases (Recall@5 = 0.650). This only shows that the index is built correctly. It is not a result on new images, since these images are in the KB.

8. Quality control

The script *audit.py* checks the outputs of all the other scripts. The saved report has 34 checks and all of them pass (Table 9).

Table 9: The 34 checks of the audit

Group	Checks	What is checked
Splits	5	The sets have the right size, no image is repeated, no image is in two sets
Index	2	The index contains only KB images
Captions	6	Five captions per image, none empty, cleaned captions match the originals, no cleaned caption for the evaluation images
Embeddings	4	Right size, no invalid number, length 1, each vector linked to the right image
Human labels	3	Every image has its entry
Images	1	All the images of the KB, validation and small test set exist and can be opened
1,000-image test set	7	1,000 different images, contains the small test set, not in the KB, not in the validation set, not in the index, five captions each, all images can be opened
Detected objects	6	Every image has its entry and the confidence scores are valid

The 11,100 images are all present and readable. The audit also found that 25 of the 1,000 reference captions have exactly the same text as a KB caption. They describe different images with very common sentences, so it is not a leakage.

9. Deliverables

At the end of Phase 1 the repository contains:

- the lists of image IDs for the KB, the validation set and the two test sets;
- the captions (original and cleaned for the KB, original only for validation and test);
- the description of the embeddings and of the two indexes;
- the objects of each image (human labels and detected objects);
- the statistics (three JSON files) and 18 figures;
- the audit report with its 34 checks;
- the scripts, a README and the list of libraries.

The images, the embeddings, the index files and the weights of the detector are too large for GitHub. They are created again when the scripts are run.

10. Limitations

- YOLOv8n was trained on COCO *train2017*, which contains our KB and validation images. It may work a little better on them than on the test images.
- The detector finds about 71% of the objects labelled by humans. A larger model would find more of them but would be slower.
- We could not test thresholds under 0.25, and 0.30 is almost as good. The choice was made on 100 images only.
- The human labels are saved only for the small test set. We need them for the 1,000 test images before the hallucination analysis of Phase 3.
- The test set is balanced only by number of objects.
- Colours, shapes, positions and context are estimated by counting words in the captions, which is approximate.
- We used only one encoder (CLIP ViT-B/32) and did not compare it with others.
- The scripts contain the path of the COCO folder on our computer (*ANN_DIR*), which has to be changed on another computer.

11. Conclusion

Phase 1 is finished. We have a knowledge base of 10,000 images, a validation set of 100 images and a test set of 1,000 images, with the captions, the embeddings, the index, the objects of each image and the statistics. The audit did not find any problem, and the test set is statistically similar to the knowledge base.

Before Phase 2 we will extract the human labels for the 1,000 test images and ask the instructors if the preprocessing is shared between the groups. In Phase 2 we will build the baseline without RAG and our RAG technique, and tune their settings on the validation set.

References

[1] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P. and Zitnick, C. L. (2014). Microsoft COCO: Common Objects in Context. ECCV 2014, 740-755.

[2] Chen, X., Fang, H., Lin, T.-Y., Vedantam, R., Gupta, S., Dollár, P. and Zitnick, C. L. (2015). Microsoft COCO Captions: Data Collection and Evaluation Server. arXiv:1504.00325.

[3] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. ICML 2021, PMLR 139, 8748-8763.

[4] Johnson, J., Douze, M. and Jégou, H. (2021). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535-547.

[5] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 33, 9459-9474.

[6] Jocher, G., Chaurasia, A. and Qiu, J. (2023). Ultralytics YOLOv8 (software). https://github.com/ultralytics/ultralytics

[7] Bird, S., Klein, E. and Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

Appendix: content of the repository

Table 10: Folders of the repository

Folder	Content
splits/	Lists of image IDs: kb_train_ids.json, val_ids.json, test_ids.json (100), test_ids_1000.json
processed/	Captions: kb_captions.json, val_references.json, test_references.json, test_1000_references.json
embeddings/	embedding_info.json, the ID files and the .npy vectors (not on GitHub)
indexes/	index_info.json and the .index files (not on GitHub)
metadata/	Human labels (kb_metadata.json, val_objects_gt.json, test_objects_gt.json) and detected objects (detected_kb.json, detected_val.json, detected_test1000.json)
reports/	dataset_stats.json, dataset_stats_detected.json, dataset_stats_extra.json, qc_report.json
figures/	The 18 figures
images/	kb/, val/, test/ (not on GitHub)

To run the project, create a virtual environment, install *requirements.txt*, download the COCO 2017 annotations, set *ANN_DIR* at the top of the scripts and run the 15 scripts in the order of Table 2.